

Start here

The one document for anyone setting up their own Hope Beacon.

This answers eight questions in order. Read it straight through if you are new. Jump to the one you need if you are not.

#	Question	Where
1	What tools does this use, and why those?	Part 1
2	What is this open-source project, and how do I take part?	Part 2
3	How do I set it up with Supabase, Vercel and Brevo?	Part 3
4	How do I put it on the internet?	Part 4
5	How do I use AI to help me?	Part 5
6	Something is broken. How do I fix it?	Part 6
7	I can code. Give me every step by hand.	Part 7
8	I cannot code. Give me the simplest path.	Part 8

You do not need to be a programmer. Part 8 is written for someone who has never opened a terminal. Part 7 is for someone who wants to know exactly what Part 8 is doing on their behalf.

Total cost: nothing. Every service below has a free tier that a normal church stays inside. Nobody is asked for a card.

Total time: about an hour, most of it waiting for things to start.

Before any of that: try it without installing anything

Open your deployed app — or clone it and run `npm run dev` — and press **"Open the tutorial"** on the front door.

You get a complete working church with sample people in it: Guides, Explorers, a Director, conversations, prayer requests, lessons, the lot. Every feature the real app has, plus a guided walk for whichever role you pick.

The tutorial is not part of the live app, and that is the point. It invents its people in your browser. It has no account, no database and no network — it works on a plane, and nothing you type into it is sent anywhere or can reach a church's real data. It is the honest way to answer "what does this actually do?" before anyone commits to setting up a database.

You can move between the two at any time with the **LIVE / TUTORIAL** control in the header. A deployment with no database configured is the tutorial and nothing else, because there is nothing else it could be.

Part 1 — The tools, and why these ones

Four services. Each one does a job the others cannot, and each was chosen for a reason you are allowed to disagree with — this is open source, and Part 2 explains how to swap any of them.

The four

Tool	Its job in one line	Free tier
GitHub	Holds the code, and how you get updates	Unlimited public repositories
Supabase	The database, and who is allowed to sign in	2 projects, 500 MB each
Vercel	Turns the code into a website people can visit	Personal projects, generous limits
Brevo	Sends the invitation emails	300 emails a day

Why Supabase

The app needs three things that normally come from three separate places: a database, a login system, and a way to say *"this Guide may read this Explorer's messages, and nobody else may."* That third one is the whole security model of a church directory, and it is the one most projects get wrong.

Supabase is PostgreSQL — a thirty-year-old database that most of the world's serious software runs on — with the login system already attached and the permission rules enforced **inside the database itself**. That last part is the reason for the choice. The rules live in `supabase/migrations/`, and they apply no matter what asks: the app, a script, a mistake, or somebody who found an API key. A rule enforced in the app is a rule that stops applying the moment somebody talks to the database directly.

It is ordinary Postgres underneath. If you leave Supabase, the tables come with you. See [BACKENDS.md](#).

Why Vercel

The app is built with Next.js, which Vercel makes. Connect your GitHub repository once and every change you push becomes a live website within about a minute, with HTTPS already set up and a real address you can give to people.

The honest version: this is convenience, not necessity. Next.js runs on Netlify, on Cloudflare, on your own server. Vercel is the shortest path from "code on GitHub" to "my church can open this on their phones", and for a church volunteer doing this on a Saturday, shortest path is the right criterion. [PLATFORMS.md](#) covers the others.

Why Brevo

Hope Beacon is invitation-only. There is no public sign-up page — the *only* way in is a Director sending someone an invitation. So sending email is not a feature, it is the front door.

Brevo sends 300 a day free, forever, with no card. A church inviting its whole congregation will not reach that in a week.

The important part is that Brevo is replaceable in one file. All the sending lives in a single function at the bottom of

church's own mail server are an edit there and nowhere else. Nothing else in the app knows which service you chose. [EMAIL.md](#) has the details.

Why not "just use Google Forms and a spreadsheet"

A fair question, and for some churches the honest answer is that a spreadsheet is enough. The line is this: a spreadsheet cannot show one Guide only their own Explorers. Everyone with the link sees everything. The moment a church is recording something a person told them in confidence, the spreadsheet is the wrong tool, and no amount of care with sharing settings fixes it.

Part 2 — The project, and how to take part

What it is

Open Hope Beacon is a free, open-source app for churches walking alongside people who want to know more about faith. One Guide, one Explorer, one conversation, and a record of it that only the right people can see.

It is licensed for you to take, change, run and keep. Fork it, rename it, strip out what you do not need, sell support for it if you like. No permission required and no strings.

Your data is yours and it never comes to us. When you set up your own Supabase project, the database is yours, in your account, under your billing. Nobody involved in this project can see inside it. There is no central server, no telemetry, no "phone home".

The repository

```
app/           the screens people see
components/    the pieces those screens are built from
lib/           the logic, including lib/live/data.ts – every database call
supabase/
  migrations/  the database, as numbered SQL files. Run in order.
  functions/   server-side code, including the invitation sender
docs/          this file and its neighbours
tests/         the checks that run before anything ships
```

How to take part

You need a free GitHub account. Nothing else.

Found a bug, or want a feature? Open an *Issue*. Go to the repository, click **Issues**, then **New issue**. Describe what you expected, what happened, and what you were doing. That is a real contribution — a clear bug report is worth more than a bad fix.

Want to change something yourself?

1. **Fork** the repository (button, top right). You now have your own copy.
2. Make your change — on GitHub's own website is fine for small edits; click any

file and press the pencil.

1. Click **Contribute** → **Open pull request**. Say what you changed and why.
2. Somebody reads it, discusses it with you, and it either goes in or it does

not. Either way you will get a reason.

Nothing you do to your fork can affect anyone else. A pull request is a *request*. This is the part people are most nervous about and it is the part with the least to fear.

Before opening a pull request, run `npm run verify:all`. It runs the same checks the project runs. If it passes locally it will almost certainly pass for everyone.

What good contributions look like here

- **Say why, not just what.** The code in this repository explains its

reasoning in comments, including the mistakes that led to it. Match that.

- **Do not break the demo.** The app must still run with no database at all —

clone, `npm run dev`, working church with sample people. `tests/no-backend.js` enforces it.

- **Never weaken a security rule to make something work.** If a rule is in your

way, say so in the pull request and let it be discussed. Rules in `supabase/migrations/` are the only thing standing between a church's private conversations and everybody.

Part 3 — Setting it up

Three accounts, in this order. Each step ends with something you can check.

Before you start

Install [Node.js](https://nodejs.org) (<https://nodejs.org>) version 22 or newer. It is a normal installer — download, next, next, done.

Check it worked. Open a terminal (**Terminal** on Mac, **PowerShell** on Windows) and type:

```
node --version
```

You want a number starting with `v22` or higher. If you get "command not found", Node is not installed — go back and run the installer.

Step 1 — Get the code

```
git clone https://github.com/klydo131/open-hope-beacon.git
cd open-hope-beacon
npm install
npm run dev
```

Open `http://localhost:3000`.

You now have a working church app with sample people in it, running entirely in your browser with no database, no accounts and no configuration. Click around. Nothing you do here can break anything.

Check: you can see a Guide's desk with Explorers on it. If you can, the hardest part of this whole document is already behind you.

Step 2 — Supabase, the database

1. Sign up at supabase.com (<https://supabase.com>). Free, no card.
2. **New project.** Give it a name and a database password — ****save that**

password somewhere**, it is not recoverable.

1. Wait about two minutes while it starts.
2. Open **Settings** → **API** and leave that page open.

Then, back in your terminal, in the project folder:

```
npm run setup
```

It asks two questions and checks your answers.

It will stop you if you paste the wrong key. That settings page shows two that look alike and sit next to each other. One is meant to go to every visitor's browser; the other bypasses every security rule you are about to create. The setup command reads which one you pasted and refuses the dangerous one. This is the single most common serious mistake and the app will not let you make it.

Step 3 — Create the tables

In Supabase, open the **SQL Editor**. Open each file in `supabase/migrations/` **in filename order**, paste the contents in, and click **Run**.

Order is not optional — each file builds on the one before.

0001_core_schema.sql	the tables, and the security rules
0001a_fix_policy_recursion.sql	a fix to those rules – must follow 0001
0002_invitations.sql	invitations
0003_invite_approval_gate.sql	who may invite whom
0004_live_api_permissions.sql	what the browser may call
0005_platform_function_acl.sql	locking down internal functions
0006_blog.sql	Guides' blog posts
0007_prayer.sql	prayer requests
0008_library.sql	shared study material
0009_meetings.sql	scheduling
0010_lock_definer_functions.sql	a security fix – see note below
0011_ministry.sql	recommendations and follow-ups
0012_lessons_and_notifications.sql	lessons and the notification bell
0013_the_invitation_is_the_approval.sql	the sign-up form, and approval on invite
20260816130240_approval_revocation_gate.sql	removing approval takes effect at once

Why 0010 exists, because it is worth your time. An earlier migration tried to take away permission to run certain internal database functions and used

to `PUBLIC`, and revoking from one member of a group does not remove what the group has. Four rounds of security testing missed it, because they all tested *tables* and this was a *function*. Supabase's own linter found it. If you ever write `revoke`, check what `PUBLIC` holds.

Restart the app: stop it with `Ctrl+C`, then `npm run dev` again.

Check: open `http://localhost:3000/setup`. That page tests your work and tells you which step you are actually on rather than making you guess.

Step 4 — Make yourself the first Director

Somebody has to be first, and they cannot be invited, because there is nobody to invite them. So the first account is made by hand, in SQL — deliberately, since the alternative is a page on the internet that hands out Executive Director to whoever finds it.

Sign up in the app with your own email. Then, in the Supabase SQL Editor, run inside it with yours.

Check: sign in. You should land on the Director's screen.

Step 5 — Brevo, so invitations can be sent

1. Sign up at [brevo.com](https://www.brevo.com) (<https://www.brevo.com>). Free, no card.
2. Verify a sender address: ****Senders, Domains & Dedicated IPs** → **Senders** → **Add a**

sender**. Brevo will not send from an address it has not confirmed is yours.

1. Create an API key: **SMTP & API** → **API Keys** → **Generate a new API key**. Copy

it — it starts `xkeysib-` and is shown once.

Turn the IP restriction OFF for this key. Brevo can limit a key to certain IP addresses. The code that sends your invitations runs on Supabase's servers, which have no fixed address, so an IP restriction blocks every invitation you ever send. The error message it produces says the credentials were rejected, which sends you looking at the key instead of the restriction.

Now give those to Supabase. In your Supabase project: **Edge Functions** → **Secrets**, and add three:

Name	Value
BREVO_API_KEY	the <code>xkeysib-...</code> key
MAIL_FROM	the sender address you just verified
SITE_URL	your app's address, e.g. <code>https://your-church.vercel.app</code>

your address.

Then deploy the invitation sender:

```
npx supabase functions deploy invite --project-ref YOUR_PROJECT_REF
```

Your project ref is in your Supabase URL: `https://<ref>.supabase.co`.

Check, and do not skip this one: invite somebody — use a second email address of your own. The message should arrive **from your church's name, not from Supabase**, and the link in it should open your app and not link to copy instead of sending it, the three secrets above are not set correctly. That is the single most common setup failure.

Part 4 — Deploying

Getting it onto the internet, where your congregation can reach it.

1. **Push your copy to your own GitHub repository.** If you forked, this is

already done.

1. Sign in at vercel.com (<https://vercel.com>) **with GitHub**.
2. **Add New** → **Project**, pick your repository, click **Import**.
3. Before deploying, open **Environment Variables** and add the same two values

`npm run setup` wrote into your `.env.local` :

```
| Name | Value | |---|---| | NEXT_PUBLIC_SUPABASE_URL | https://<your-ref>.supabase.co ||  
NEXT_PUBLIC_SUPABASE_ANON_KEY | the long eyJ... publishable key |
```

> **Not the service_role key. Ever.** Anything named `NEXT_PUBLIC_` is sent > to every visitor's browser. That is fine for the publishable key — it is > designed for it, and your security rules are what protect your data. The > `service_role` key bypasses every one of those rules.

1. **Deploy.** About a minute.
2. Copy your new address and set it as `SITE_URL` in Supabase's Edge Function

secrets (Part 3, Step 5), then redeploy the invite function.

1. In Supabase, **Authentication** → **URL Configuration**, set **Site URL** to your address and add `https://your-address/join` to **Redirect URLs**.

From now on, every push to your `main` branch updates the live site automatically. There is no second step and no "publish" button.

Check: open your address on your phone, on mobile data with wifi off. Sign in. If it works there, it works.

Part 5 — Using AI to do the work

You can hand most of this document to an AI assistant and have it do the typing. This section is about doing that *well*, because the failure modes are specific.

What to use

Any of Claude, ChatGPT, Gemini, GitHub Copilot, Cursor. The free tiers are enough for setup. There is a longer list in [AI-TOOLKIT.md](#).

The one thing that matters most

Give it the repository, not a description of the repository.

An AI that has read `supabase/migrations/0001_core_schema.sql` knows your exact security rules. One that has only been told "it's a church app with Supabase" will invent a plausible schema that does not match yours, and everything it writes afterwards will be subtly wrong in a way that is hard to see.

Tools like Claude Code, Cursor and Copilot Workspace read the actual files. Use one of those if you can. If you are using a chat window, paste the real file.

Prompts that work

Setting up:

I am setting up Open Hope Beacon from `github.com/klydo131/open-hope-beacon`. Read `docs/START-HERE.md` and `docs/SETUP.md`. I am on Part 3 Step 3 and the SQL editor returned this error: `<paste the exact error>`. What went wrong and what do I run instead?

Making a change:

In this repository, I want Explorers to be able to mark a lesson as finished. Before writing anything: read `supabase/migrations/0012_lessons_and_notifications.sql` and `lib/live/data.ts`, and tell me which files you would change and why. Do not write code yet.

That last sentence is the important one. Ask for the plan first. It takes ten seconds to read a plan and a long time to unpick a confident wrong change.

Understanding something:

Explain what `supabase/migrations/0003_invite_approval_gate.sql` does, in plain English, as if I do not know SQL. What could go wrong if I removed it?

The rules, and they are not optional

Never paste a key into a chat window. Not the `service_role` key, not the Brevo key, not your database password. Say `<my API key>` instead. If you have already pasted one, go and rotate it now — it is a button in the same place you created it.

Make it show you, not tell you. "Did that work?" gets you a cheerful yes. "Run the query that proves it and show me the output" gets you the truth. This document exists partly because an AI reported a fixed invitation system twice before anyone clicked a link.

Ask for the negative test. If it says a security rule works, ask it to prove the rule *blocks* what it should block. A test where nothing fails has proved nothing. Every security fix in this project is checked that way, and the reason is that a test suite once passed while refusing four things for a completely unrelated reason.

It will be confidently wrong sometimes. Not occasionally — regularly, and most convincingly on the things it half-knows. Check anything that touches money, security, or somebody's private conversation.

Part 6 — When something breaks

Ordered by how often they actually happen.

"The invitation email never arrives"

Almost always the three Edge Function secrets. Check `BREVO_API_KEY`,

Tell the two failures apart: if the app says the invitation was created and offers you a link to copy by hand, the function ran fine and the *sending* failed. It always tells you why in the same message. If it is Brevo's IP allow-list, switch the restriction off — Supabase's servers have no fixed address.

Read the actual error rather than guessing: Supabase → **Edge Functions** → **invite** → **Logs**.

"The invitation link says invalid or expired"

If you are running a copy from before 18 August 2026, update it. There was a real bug: the link carried the wrong one of the three tokens Supabase issues, so *every* invitation failed on arrival for everyone, and the message blamed expiry. Pull the latest `main`.

If you are up to date, check `SITE_URL` matches your real address, and that

Configuration → Redirect URLs**.

Note that invitation links genuinely do work only once. If you clicked it to test, the next click will fail correctly.

"I signed in and it sent me straight back to the sign-in page"

Your account exists but is not approved. Somebody with a Director account must approve you — or, if you are the first person, you have not run

"It works on my computer but the deployed site is blank"

The environment variables are missing on Vercel. `.env.local` is on your machine and deliberately never uploaded. Add them in Vercel's project settings (Part 4, Step 4) and redeploy — **changing them does not redeploy by itself**.

"Permission denied" or "row violates row-level security"

The security rules are working; the account asking does not have the right. This is the system doing its job, not a bug. Check the account's role and that it is approved. **Do not "fix" this by turning off row level security** — that makes every private conversation in your church readable by anyone with your publishable key, which is in every visitor's browser.

"Migration failed"

Read the error; Postgres is unusually specific. The two common causes are running files out of order, or running one twice. Most are written to be safe to re-run; if one is not, the error will say the object already exists, which is harmless.

Nothing above matches

1. `npm run verify:all` — it checks a lot and names what it finds.
2. Supabase → **Logs** for database errors, **Edge Functions** → **Logs** for email.
3. Your browser's console (`F12`) for anything on screen.
4. Open an Issue on GitHub with the exact error text. Somebody has probably hit

it.

Part 7 — The manual path, for people who code

Everything Part 8 does for you, done by hand. Assumes you are comfortable with a terminal, git, and SQL.

```
# 1. Code
git clone https://github.com/klydo131/open-hope-beacon.git
cd open-hope-beacon && npm install

# 2. Runs with no backend at all – this is the design, not a fallback
npm run dev          # localhost:3000, sample data, in-browser only

# 3. Point it at your own Supabase project
cat > .env.local <<'EOF'
NEXT_PUBLIC_SUPABASE_URL=https://<ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<publishable key, the eyJ... one>
EOF
# or: npm run setup – same result, and it refuses the service_role key

# 4. Schema, in filename order. Every file, no exceptions.
for f in supabase/migrations/*.sql; do
  psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$f"
done

# 5. Bootstrap yourself, after signing up in the app with this address
psql "$DATABASE_URL" -f supabase/seed/01_make_me_the_first_director.sql

# 6. Email
npx supabase secrets set \
  BREVO_API_KEY=xkeysib-... \
  MAIL_FROM=invites@yourchurch.org \
  SITE_URL=https://yourchurch.vercel.app \
  --project-ref <ref>
npx supabase functions deploy invite --project-ref <ref>

# 7. Prove it before you trust it
npm run verify:all
npm run build
```

What to read, in this order

File	Why
<code>lib/mode.ts</code>	<code>IS_LIVE / IS_DEMO</code> . The whole switch is the presence of the two keys.
<code>lib/live/data.ts</code>	Every database call the browser makes. One file on purpose.
<code>supabase/migrations/0001_core_schema.sql</code>	Tables and the security rules. Read the comments.
<code>supabase/functions/invite/index.ts</code>	The only code holding the <code>service_role</code> key.
<code>app/api/auth/sign-in/route.ts</code>	Sign-in runs server-side so credentials never touch client code.

Things that will bite you

- **Contracts and migrations move forward only.** Never edit an applied

migration; add a new numbered one. Somebody else's database already ran the old one.

- `revoke ... from anon` **may do nothing**. If the grant went to `PUBLIC` ,

revoking from one role changes nothing. See `0010_lock_definer_functions.sql` and check `PUBLIC` first.

- `?code=` **is not a generic word for "token"**. In `@supabase/supabase-js` it

selects the PKCE route, which needs a verifier the browser wrote when it *started* the flow. A server-minted link has none, so it fails on every device. Emailed links must carry `token_hash` and redeem with `verifyOtp` .

- **A CHECK constraint may only call IMMUTABLE functions.** `current_date` is

not one. Postgres refuses it, and it is right to.

- **Deploying an Edge Function is not committing it.** They are separate

actions and this project has twice had a function running live with no source in the repository.

- `tests/no-backend.js` **is a real constraint**. The app must still run with no

database. Adding a hard requirement on an env var breaks the build.

Part 8 — The simple path, for everyone else

You have never used a terminal. That is fine. Follow this exactly.

Every step tells you what you should see. If you do not see it, stop there — the step after will not fix it — and check Part 6.

What you will have at the end

A website at your own address that your church signs into on their phones.

Step 1 — Install Node.js (5 minutes)

Go to nodejs.org (<https://nodejs.org>) and download the big green **LTS** button. Run the installer, click next until it finishes.

Step 2 — Open a terminal (1 minute)

- **Windows:** Start menu, type `powershell`, press Enter.
- **Mac:** `Cmd+Space`, type `terminal`, press Enter.

A window with text appears. You type commands here and press Enter. It cannot break your computer.

Type this and press Enter:

```
node --version
```

You should see something like `v22.14.0`. If it says "not recognized", Node did not install — go back to Step 1.

Step 3 — Download the app (3 minutes)

Type these one at a time, pressing Enter after each and waiting for it to finish:

```
git clone https://github.com/klydo131/open-hope-beacon.git
```

```
cd open-hope-beacon
```

```
npm install
```

The last one prints a lot and takes a minute or two. Warnings are normal.

Step 4 — See it working (1 minute)

```
npm run dev
```

Open your web browser and go to `http://localhost:3000`.

You should see Hope Beacon with sample people in it. **This is the whole app.** Click around — it is not connected to anything and you cannot break it.

Leave this window running. To stop it later, press `Ctrl+C`.

Step 5 — Make a database (10 minutes, mostly waiting)

1. Go to supabase.com (<https://supabase.com>), **Start your project**, sign in with GitHub.

1. **New project**. Name it after your church. Set a database password and **write it down** — you cannot get it back.

1. It takes about two minutes to start. Wait.
2. Click **Settings** (gear, bottom left) → **API**. Leave this page open.

Step 6 — Connect them (3 minutes)

Open a **second** terminal window (leave the first running). Then:

```
cd open-hope-beacon
```

```
npm run setup
```

It asks two questions. Copy the answers from the Supabase page you left open.

If it says you pasted the wrong key, it is right and you did. That page shows two long keys that look almost identical. You want the **publishable** or **anon** one, not the one labelled `service_role`. This check exists because that mistake would expose everything.

Step 7 — Build the tables (15 minutes)

This is the longest step. It is repetitive, not hard.

In Supabase, click **SQL Editor** in the left sidebar.

On your computer, open the `open-hope-beacon` folder, then `supabase`, then

For each file, in order, top to bottom:

1. Open it (any text editor — Notepad, TextEdit).
2. Select all (`Ctrl+A` / `Cmd+A`), copy (`Ctrl+C` / `Cmd+C`).
3. In Supabase's SQL Editor, paste, click **Run**.
4. Wait for "Success". Then the next file.

The order matters and is not optional. Each file builds on the one before.

You should see "Success. No rows returned" for most of them. That is what success looks like here.

Step 8 — Restart (1 minute)

Go back to the first terminal window. Press `Ctrl+C`. Then:

```
npm run dev
```

Check: go to `http://localhost:3000/setup`. That page tests everything you just did and tells you if anything is missing.

Step 9 — Make yourself the Director (5 minutes)

At `http://localhost:3000`, sign up with your real email and a password.

Then open `supabase/seed/01_make_me_the_first_director.sql`. Find the email address inside and replace it with yours. Copy the whole file into Supabase's SQL Editor and click **Run**.

Sign out of the app and sign back in.

You should see the Director's screen, with the whole church on it.

Step 10 — Put it on the internet (10 minutes)

1. Go to vercel.com (<https://vercel.com>) and sign in with **GitHub**.
2. **Add New** → **Project**. Find `open-hope-beacon` and click **Import**.
3. Before clicking Deploy, expand **Environment Variables** and add two. They

are in the `.env.local` file that Step 6 created in your project folder — open it and copy both lines.

1. Click **Deploy** and wait about a minute.
2. It gives you an address like `open-hope-beacon-xyz.vercel.app`. **That is your

app.** Open it on your phone.

Step 11 — Turn on invitations (10 minutes)

Nobody else can join until this is done.

1. Sign up at [brevo.com](https://www.brevo.com) (<https://www.brevo.com>) — free, no card.
2. **Senders, Domains & Dedicated IPs** → **Senders** → **Add a sender**. Use a church

address. Confirm it from the email they send you.

1. **SMTP & API** → **API Keys** → **Generate a new API key**. Copy it now; it is shown once. **If it offers to restrict the key to an IP address, say no** — it will block every invitation you send.

1. In Supabase: **Edge Functions** → **Secrets**, add three:

- `BREVO_API_KEY` — the key you just copied
- `MAIL_FROM` — the sender address you verified
- `SITE_URL` — your Vercel address, starting `https://`

1. In Supabase: **Authentication** → **URL Configuration**. Set **Site URL** to your

Vercel address, and add `https://your-address/join` under **Redirect URLs**.

Check, and actually do it: in your app, invite a second email address of your own. The message should arrive **from your church**, and its link should open your app. If it does, you are finished — invite your Directors and Guides, and they invite everyone else.

You are done

Keep somewhere safe: your Supabase database password, your Brevo API key, and your app's address.

To update later: in GitHub, open your fork, click **Sync fork**. Vercel rebuilds automatically. If an update adds a migration, run it the way you did in Step 7.

Where to go next

Document	What it covers
SETUP.md	The short version of Part 3
BUILD-YOUR-OWN.md	Why the backend is shaped this way
BACKENDS.md	Using something other than Supabase
PLATFORMS.md	Hosting somewhere other than Vercel
EMAIL.md	Using something other than Brevo
SECURITY.md	The rules, and how to check them yourself
ONBOARDING.md	Getting a real church using it
AI-TOOLKIT.md	Longer version of Part 5
UPDATES.md	Keeping your copy current

Stuck? Open an Issue at

what you expected, and the exact error. That is a contribution too.